

CENTRO DE INFORMÁTICA — UFPB

ENGENHARIA DE REQUISITOS (ER)

NA05 — MÉTODOS ÁGEIS DE DESENVOLVIMENTO

Danielle Rousy
danielle@ci.ufpb.br



O QUE VEREMOS NESSA AULA?

Métodos ágeis de desenvolvimento

- XP e Scrum



XP (EXTREME PROGRAMMING)

Projeto C3 (Chrysler) - Kent Beck, Ward Cunningham and Ron Jeffries (1996)

- Disponível em: <http://www.xprogramming.org>

Baseado em princípios ágeis.

Valores:

- Comunicação
- Simplicidade
- Feedback
- Coragem
- Respeito

12 Práticas:

*Pair Programming, Refactoring, Simple Design, Test-driven development
Collective Ownership, Coding Standard, Continuous Integration, Sustainable Pace
Customer tests, Whole Team, Planning Game, Small Releases, Metaphor*

COMUNICAÇÃO

Várias práticas do XP promovem uma maior comunicação entre os membros da equipe

A comunicação não é limitada por procedimentos formais.

- Usa-se o melhor meio possível, que pode ser
- Uma conversa ou reunião informal;
- Um e-mail, um bate-papo, um telefonema;
- Diagramas, se necessário (pode, mas não precisa, ser UML);
- O próprio código;
- "Estórias" elaboradas pelo usuário-final;
- ...

SIMPLICIDADE

XP incentiva ao extremo práticas que reduzam a complexidade do sistema

- A solução adotada deve ser sempre a mais simples que alcance os objetivos esperados;
- Use as tecnologias, design, algoritmos e técnicas mais simples que permitirão atender aos requerimentos do usuário-final;
- Design, processo e código podem ser simplificados a qualquer momento;
- Qualquer design, processo ou código criado pensando em iterações futuras deve ser descartado.

FEEDBACK

Várias práticas do XP garantem um rápido feedback sobre várias etapas/partes do processo

Feedback sobre qualidade do código (testes de unidade, programação em pares, posse coletiva)

Feedback sobre estado do desenvolvimento (estórias do usuário-final, integração contínua, jogo do planejamento)



CORAGEM

Testes, integração contínua, programação em pares e outras práticas do XP aumentam a confiança do programador e ajudam-no a ter coragem para:

- melhorar o design de código que está funcionando para torná-lo mais simples
- jogar fora código desnecessário
- investir tempo no desenvolvimento de testes
- mexer no design em estágio avançado do projeto
- pedir ajuda aos que sabem mais
- dizer ao cliente que um requisito não vai ser implementado no prazo prometido
- abandonar processos formais e fazer design e documentação em forma de código

XP

12 Práticas:

Pair Programing, Refactoring, Simple Design, Test-driven development, Collective Ownership, Coding Standard, Continuous Integration, Sustainable Pace, Customer tests, Whole Team, Planning Game, Small Releases, Metaphor



Fonte: Devmedia. Disponível em: <http://www.devmedia.com.br/agile-development-xp-e-scrum-em-uma-abordagem-comparativa/30808>

XP



Codificação é a atividade central do projeto



Testes (que também é código) servem de especificação



Comunicação entre desenvolvedores se baseia em código

O MANTRA DO PROGRAMADOR XP



Codifique, senão o software não sai



Teste, senão você não sabe se está funcionando



Refatore, senão o código vai ficar tão ruim que será impossível dar manutenção



Escute, para que saiba qual é o problema a resolver

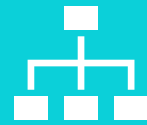


Planeje, para que você sempre faça a coisa mais importante ainda a fazer

REFATORAMENTO



Refatorar é melhorar o código sem alterar sua funcionalidade



Antes de fazer uma mudança, você refatora o código para que a mudança seja simples de fazer



Refatoração continua possibilita manter um design legal, mesmo com mudanças freqüentes

TESTES AUTOMÁTICOS

Testes automáticos são parte do software

- Se você tem somente a funcionalidade, seu software está incompleto

Testes permitem que você refatore sem medo de quebrar o código

Testes representam uma “redundância lógica” que você adiciona ao código

Escrevendo testes **antes** da funcionalidade, você clareia dúvidas sobre o que o software deve fazer

Testes de unidade x testes funcionais

ALGUMAS CONSIDERAÇÕES SOBRE TESTES UNITÁRIOS

Os testes unitários procuram aferir a corretude do código, **em sua menor fração**

```
1  function soma(a, b){  
2    return a + b;  
3  }
```

unit_test_example_1.js hosted with ❤ by GitHub

[view raw](#)

```
1  var resultado = soma(1, 2);  
2  expect(resultado).toEqual(3);
```

uniqu_test_example_2.js hosted with ❤ by GitHub

[view raw](#)

OUTRO EXEMPLO

```
public class OperacoesPessoa
{
    private IPessoaRepositorio pessoaRepositorio;

    public OperacoesPessoa(IPessoaRepositorio pessoaRepositorio)
    {
        this.pessoaRepositorio = pessoaRepositorio;
    }

    public int ObterIdadePessoa(string cpf)
    {
        var pessoa = pessoaRepositorio.ObterPessoa(cpf);

        var dataAtual = DateTime.Today;

        var idade = dataAtual.Year - pessoa.Nascimento.Year;

        if (pessoa.Nascimento > dataAtual.AddYears(-idade))
            idade--;

        return idade;
    }
}
```

```
[TestClass]
public class TestePessoa
{
    [TestMethod]
    public void TestarIdade()
    {
        var nascimento = new DateTime(2018, 1, 1);
        var repositorioMockPessoa = PessoaRepositorioMock.Instancia().PreencherObterPessoa(nascimento).Mock();

        var operacoesPessoa = new OperacoesPessoa(repositorioMockPessoa.Object);

        var idade = operacoesPessoa.ObterIdadePessoa("123.456.789-00");

        Assert.IsTrue(idade == 0);
    }

    [TestMethod]
    public void TestarIdade2()
    {
        var nascimento = new DateTime(1994, 12, 23);
        var repositorioMockPessoa = PessoaRepositorioMock.Instancia().PreencherObterPessoa(nascimento).Mock();

        var operacoesPessoa = new OperacoesPessoa(repositorioMockPessoa.Object);

        var idade = operacoesPessoa.ObterIdadePessoa("123.456.789-00");

        Assert.IsTrue(idade == 23);
    }
}
```

POR QUE ESCREVER OS TESTES?

Ao criar **classes de testes** separamos a verificação da corretude do código das regras de negócio da aplicação, garantindo que este processo não comprometa, de alguma forma, código principal.

Considerando que o código de sua aplicação só tende a aumentar, ter rotinas de testes separadas para garantir que o código continua funcionando agiliza muito o desenvolvimento.

FERRAMENTAS DE TESTE



PHPUnit



Jasmine

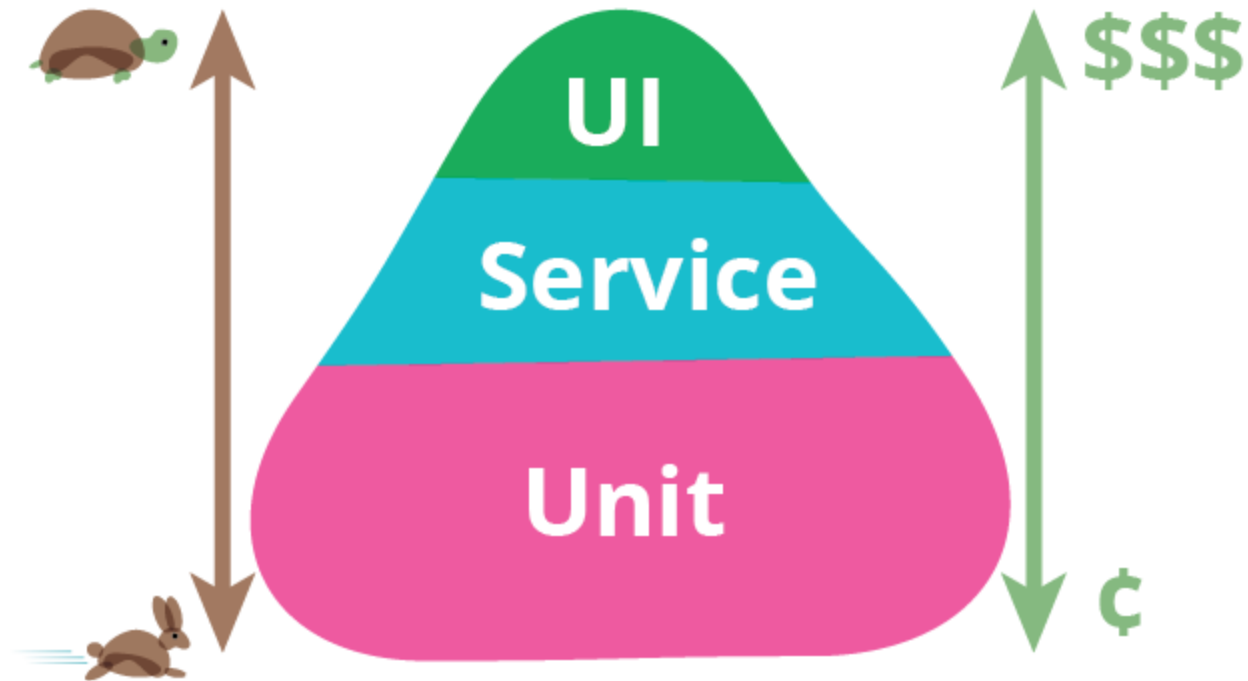
JUnit



simple, flexible

fun

TESTES UNITÁRIOS



O DESIGN MAIS SIMPLES POSSÍVEL

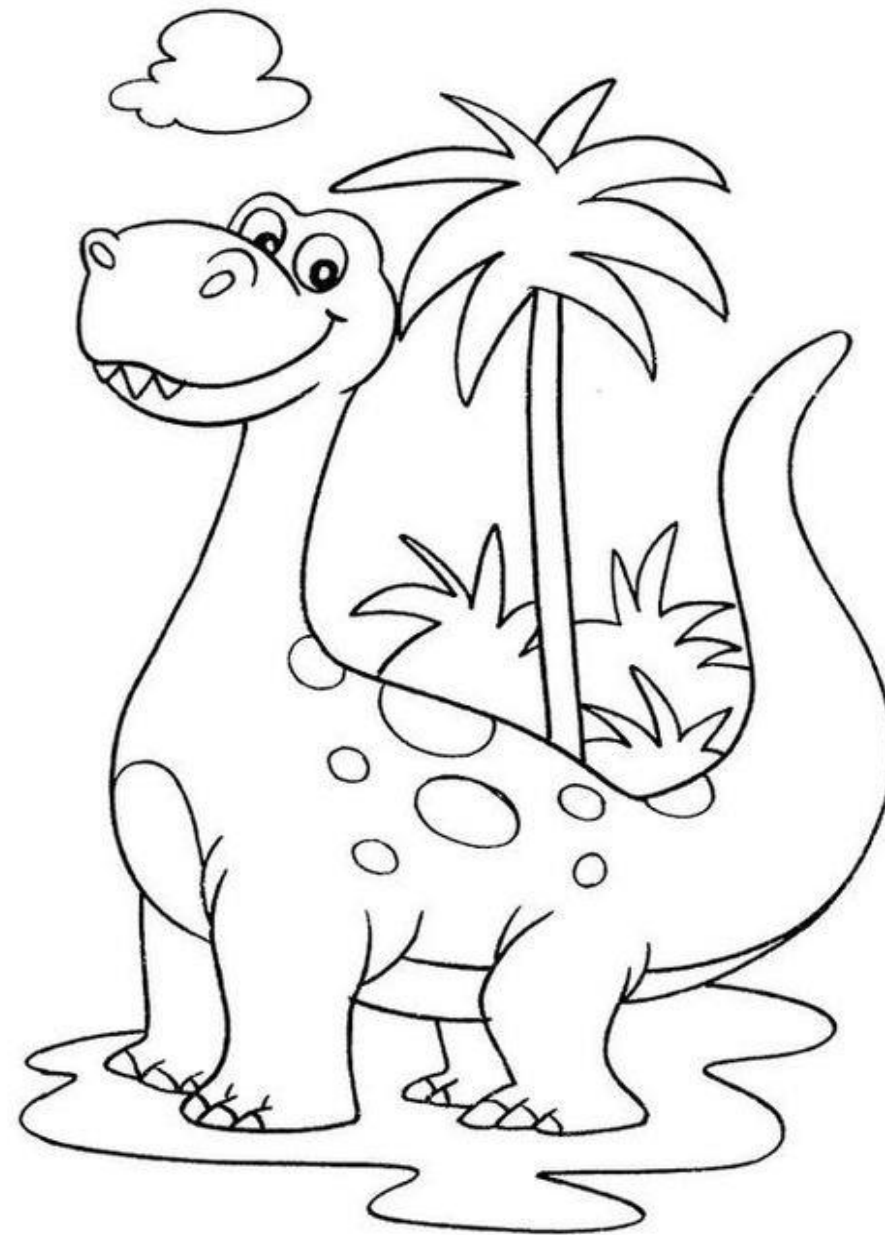
Designs flexíveis são uma defesa contra mudanças imprevistas no software

Porém, designs flexíveis também têm custos

- Tempo para desenvolvimento e manutenção
- O código fica mais complexo
- Muitas vezes a flexibilidade não é utilizada nunca

Como mudança é barata em XP, vamos manter o design mais simples possível, modificando-o quando for necessário suportar mais funcionalidade

VAMOS FAZER UM
PEQUENO EXERCÍCIO...



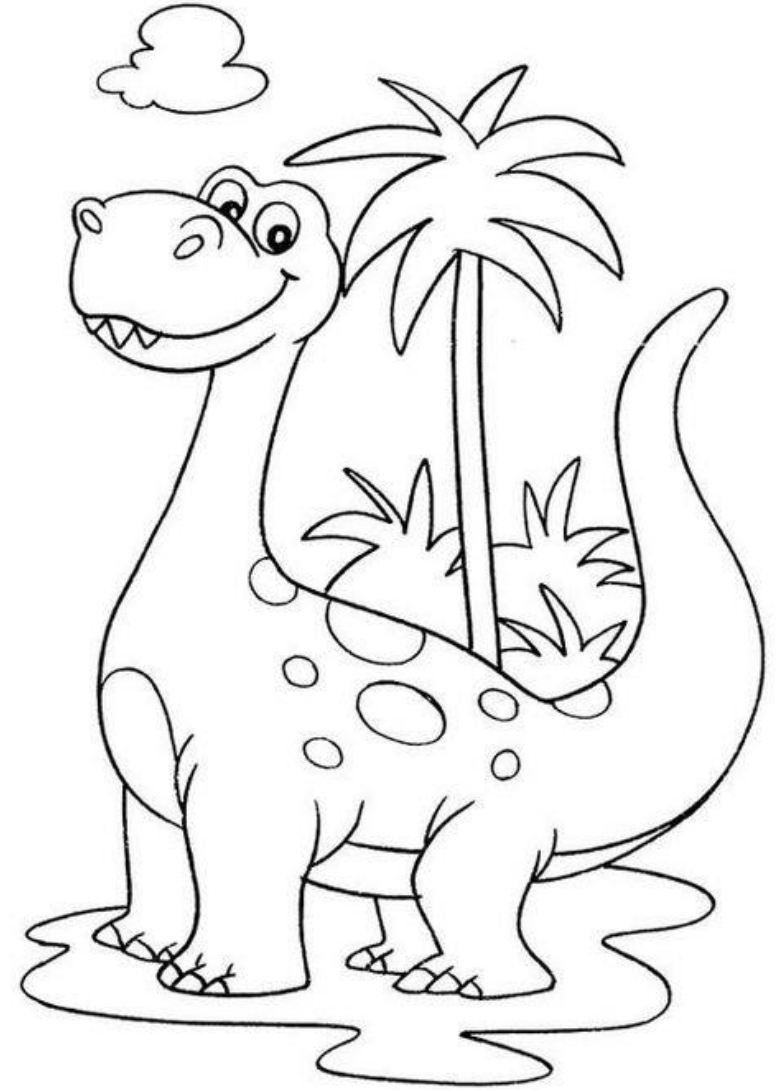
VAMOS FAZER UM PEQUENO EXERCÍCIO...

Forme pares

Em 18min faça o desenho segundo o espelho mostrado abaixo. Um dos pares ficará como o desenhista, e o outro como revisor.

A cada 3min, o papel é alterado.

Um dos intervalos será usado para debater melhorias na realização.



PAIR PROGRAMMING

“Programadores, se deixados por conta própria, ignoram os erros mais gritantes, os quais qualquer pessoa seria capaz de detectar instantaneamente”.

Isso se deve a um princípio bastante estudado na psicologia chamado **dissonância cognitiva**.

Temos a tendência natural, como programadores, de acreditar que aquilo que escrevemos está certo.

PAIR PROGRAMMING IS...

TWO programmers working side-by-side, collaborating on the same design, algorithm, code or test. One programmer, the driver, has control of the keyboard/mouse and actively implements the program. The other programmer, the observer, continuously observes the work of the driver to identify tactical (syntactic, spelling, etc.) defects and also thinks strategically about the direction of the work. On demand, the two programmers can brainstorm any challenging problem. Because the two programmers periodically switch roles, they work together as equals to develop software.

--Laurie Williams
North Carolina State University Computer

Science

Source: <http://www.pairprogramming.com/>

PROGRAMANDO EM PARES

Se revisão de código é legal, vamos fazê-la o tempo todo

Em XP, programação é feita em pares

Pares mudam com relativa rapidez (em dias)

Programação em pares favorece comunicação e aprendizado

Mas, você precisa estabelecer um padrão de codificação

Há casos de **redução** no tempo de desenvolvimento com programação em pares

ALGUMAS DIFICULDADES

Vencer barreiras culturais

- Deixar alguém mexer no seu código
- Trabalhar em pares
- Ter coragem de admitir que não sabe
- Pedir ajuda

Vencer hábitos antigos

- Manter as coisas simples (e não tentar prever o futuro escrevendo "design flexível")
- Jogar fora código desnecessário
- Escrever testes antes de codificar
- Refatorar com frequência (vencer o medo)

QUANDO NÃO USAR?

Equipes grandes e espalhadas geograficamente

- Comunicação é um valor fundamental do XP
- Não é fácil garantir o nível de comunicação requerido em projetos XP em grandes equipes

Situações onde não se tem controle sobre o código

- Código legado que não pode ser modificado(XP tem dificuldade em reaproveitar código) RUIMMM

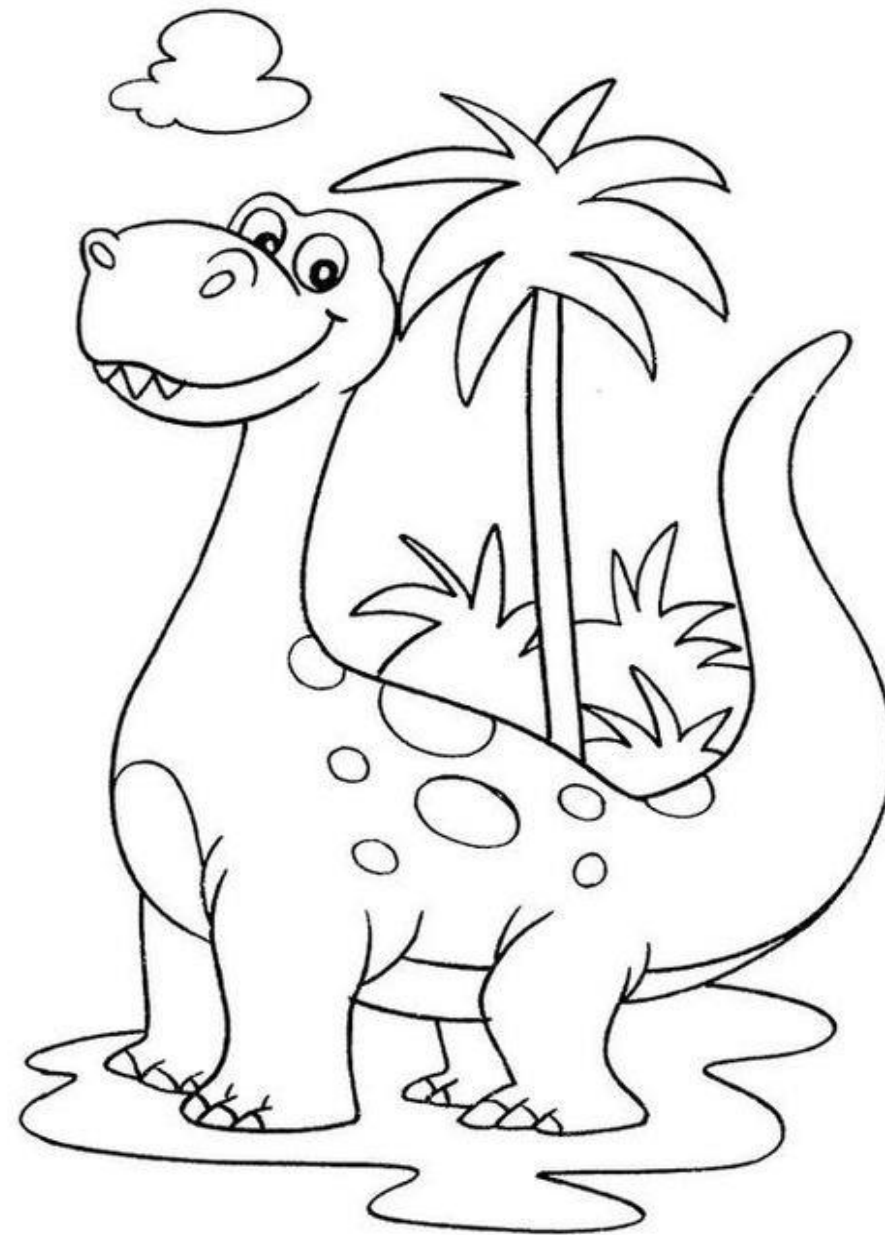
Situações onde o feedback é demorado

- compile-link-build-test que leva 24 horas
- testes muito difíceis, arriscados e que levam tempo

PAUSA PARA RECONCENTRAR!



VAMOS FAZER UM
PEQUENO EXERCÍCIO...



1



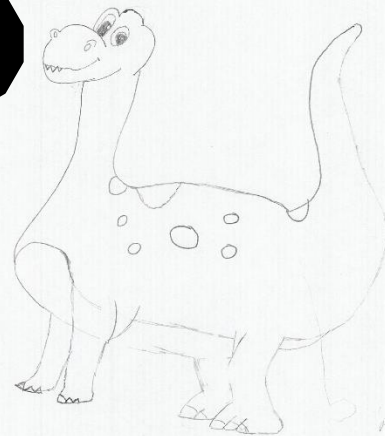
2



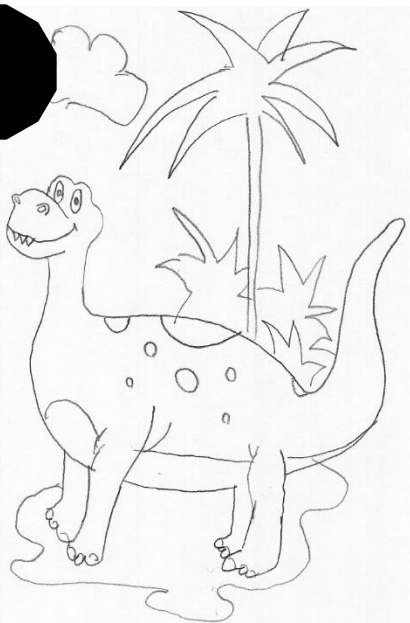
3



4



5



6



7



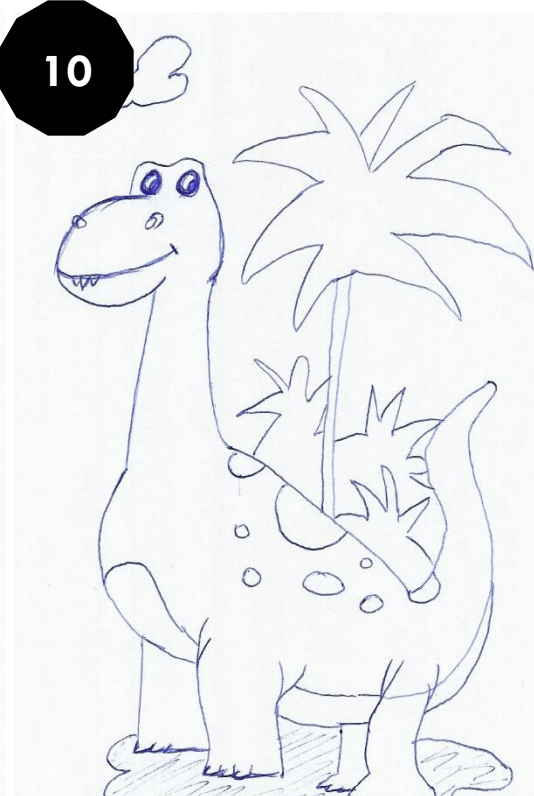
8



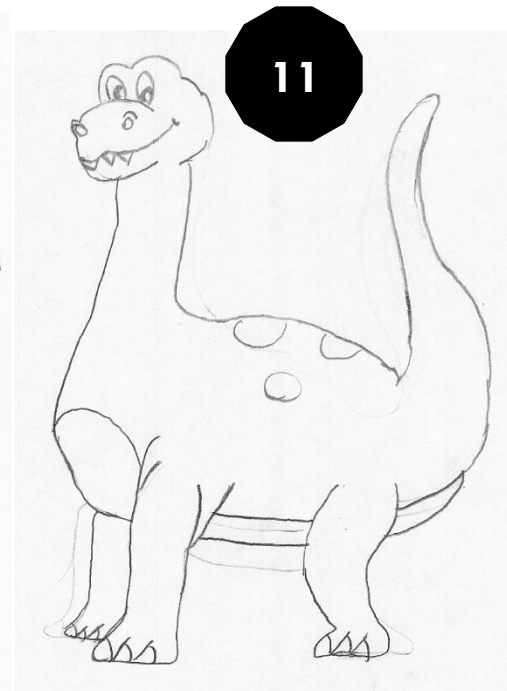
9



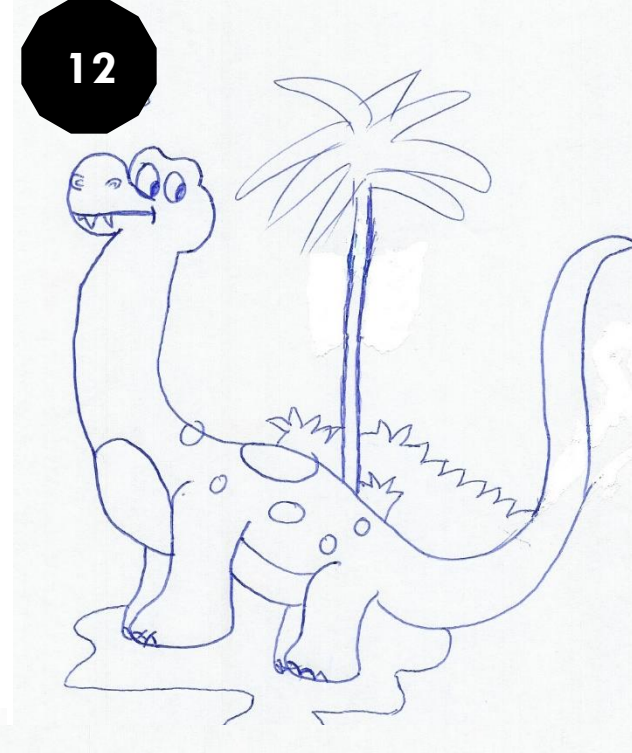
10



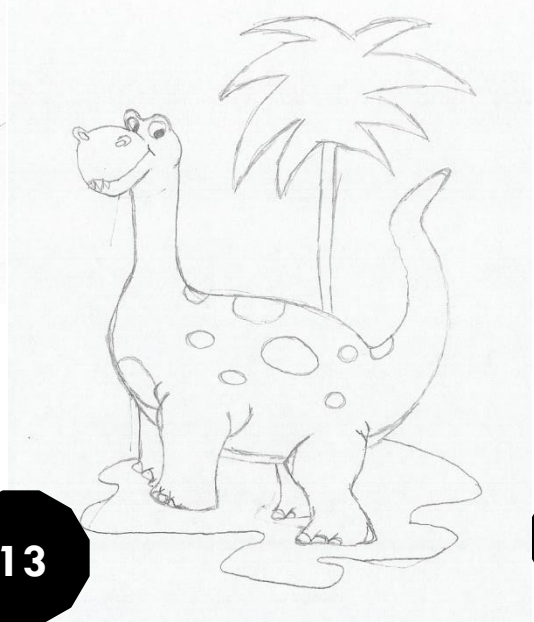
11



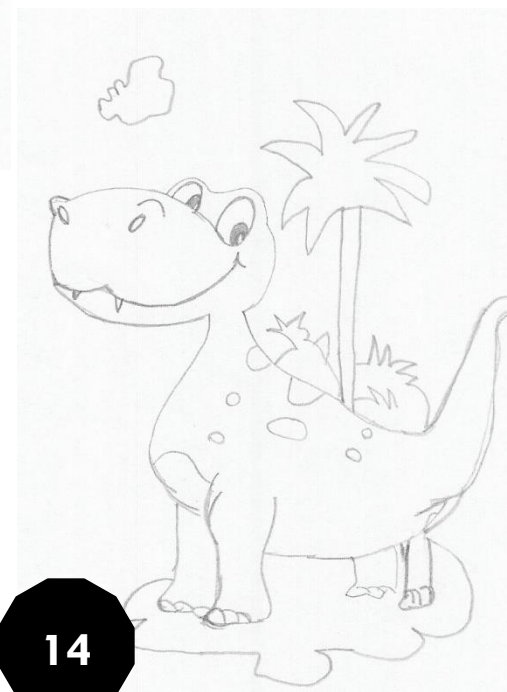
12



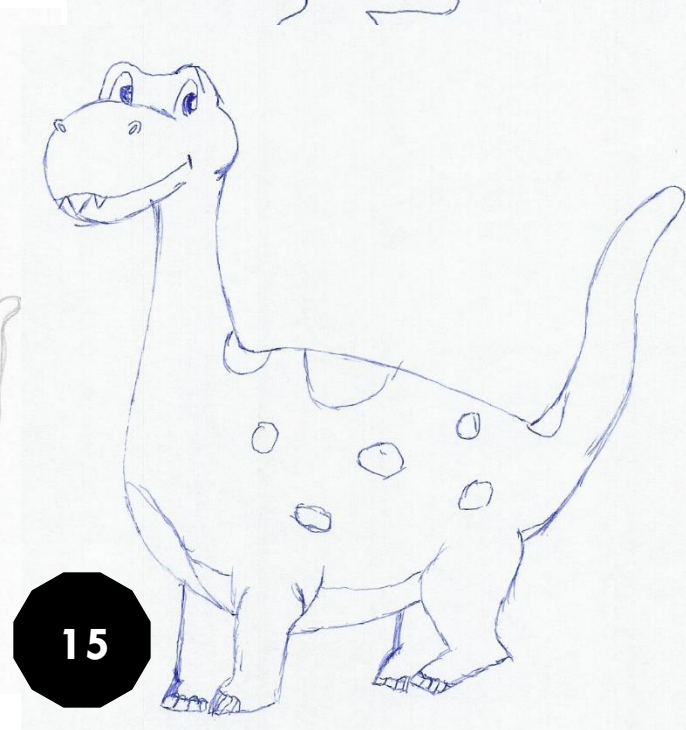
13



14



15



16



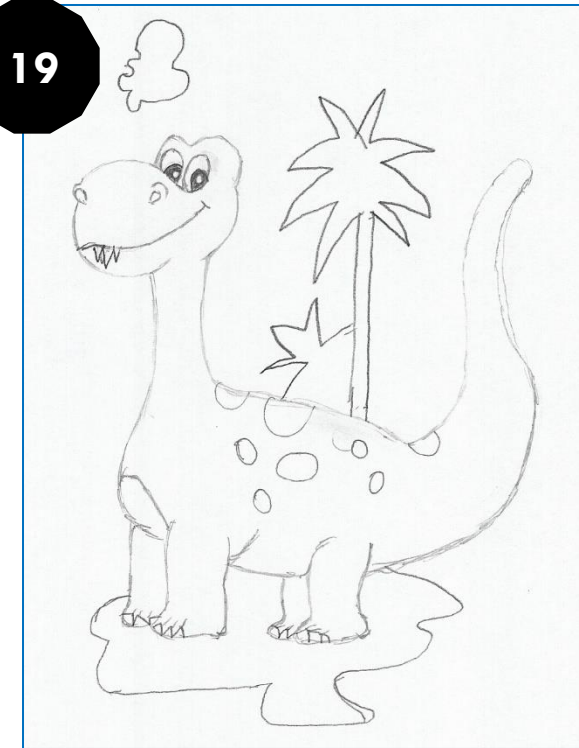
17



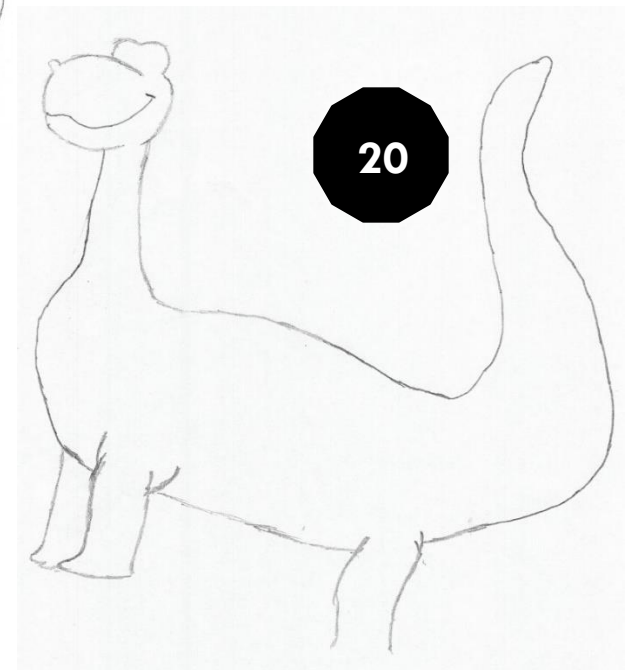
18



19



20



PROGRAMAÇÃO EM PARES

Duas pessoas trabalhando juntas no mesmo código em um único computador – piloto e o navegador

Piloto tem a função de escrever o código

Navegador faz a inspeção do trabalho



SCRUM - COMO TUDO COMEÇOU...

SCRUM - História

- *Inspiração: Takeuchi and Nonaka in "The new new product development game" (Harvard Business Review, Jan-Feb 1986)*
- *Começou em 1990*
- *Primeira implementação 1993 - Sutherland, Scumnotales and McKenna*
- *Apresentado na OOPSLA 95 - Jeff Sutherland e Ken Schwaber, "The Scrum Development Process"*

Manifesto Ágil - Introdução

- *Criado em 2001*
- *Melhorar o desenvolvimento considerando as seguintes prioridades:*
 - *Indivíduos e Interações X Processos e Ferramentas*
 - *Software funcionando X Documentação Extensa*
 - *Colaboração com o Cliente X Negociação de Contrato*
 - *Responder a Mudanças X Seguir Planos*

SCRUM - DEFINIÇÃO

- projetos usando equipes pequenas e multidisciplinares (*cross-functional*) produziram os melhores resultados
- é um método de reinício de jogada no rugby
- vem do inglês *scrimmage* (escaramuça - manobra derradeira que visa mudar o fim de algo).

É um framework
simples
para gerenciar projetos
COMPLEXOS



SCRUM - CARACTERÍSTICAS



não descreve o que fazer em cada situação.

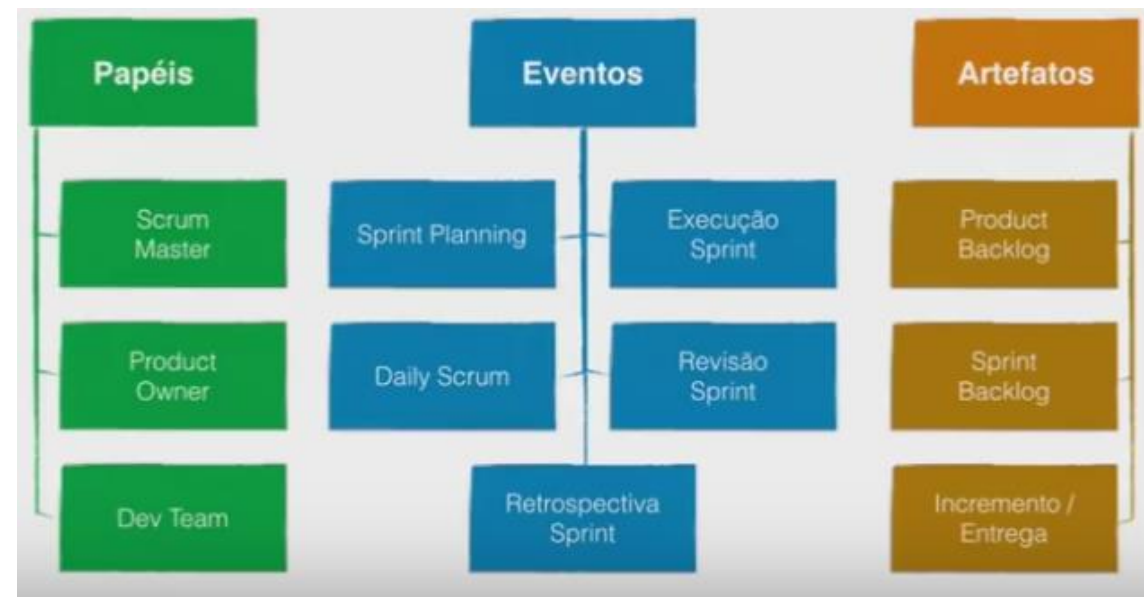


usado para trabalhos complexos nos quais é impossível prever tudo o que irá ocorrer.



Apesar ter sido destinado para gerenciamento de projetos de software, ele pode ser utilizado em equipes de manutenção de software ou como uma abordagem geral de gerenciamento de projetos/programas.

SCRUM - PRÁTICAS FUNDAMENTAIS



SCRUM - PAPÉIS



Dev Team

- multifuncional
- entre 5 e 9 pessoas
- decidem como fazer
- tempo integral
- auto organizável
- troca só na mudança de sprint

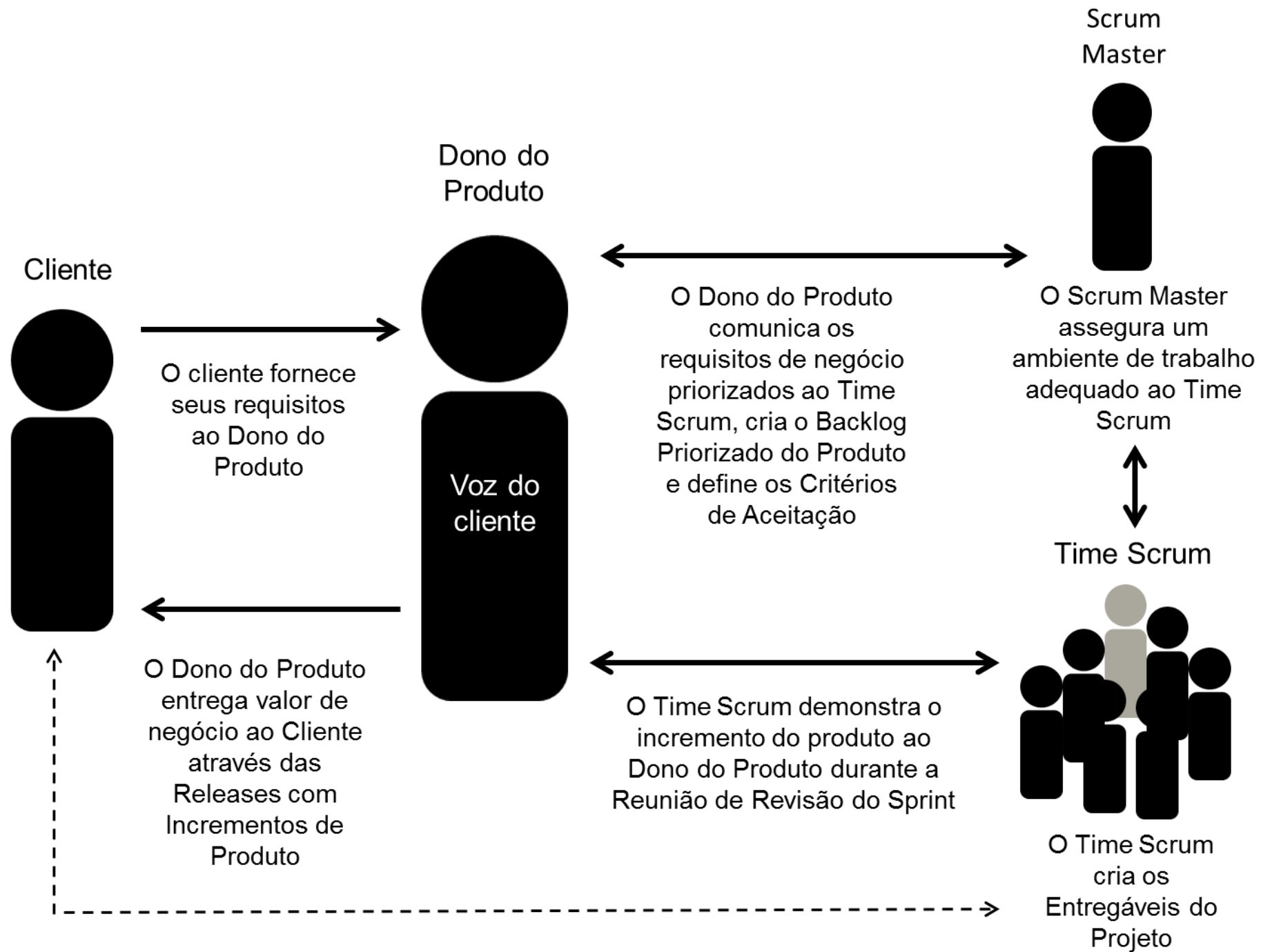
- decide recursos, funcionalidades
- decide datas de lançamento e conteúdo
- prioriza itens product backlog de acordo com o valor de mercado
- responsável pela rentabilidade (ROI)
- aceita/rejeita resultados

Product Owner



Scrum Master

- coach, líder, facilitador, não é chefe
- remove obstáculos
- funcionalidade/produtividade da equipe
- colaboração entre os diversos papéis e funções
- escudo para interferências externas





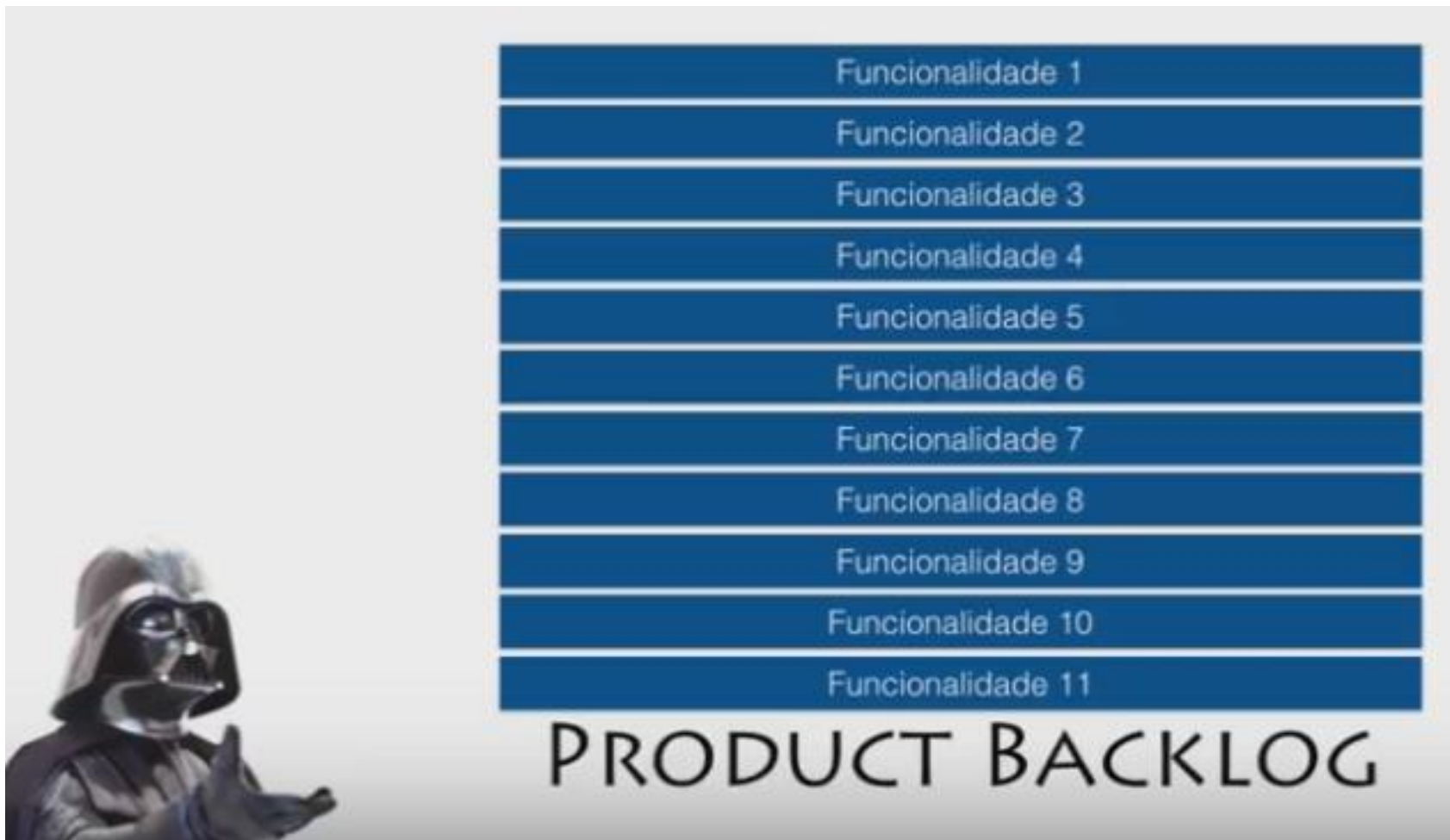
VISÃO GERAL

- o que quer
- aonde quer chegar

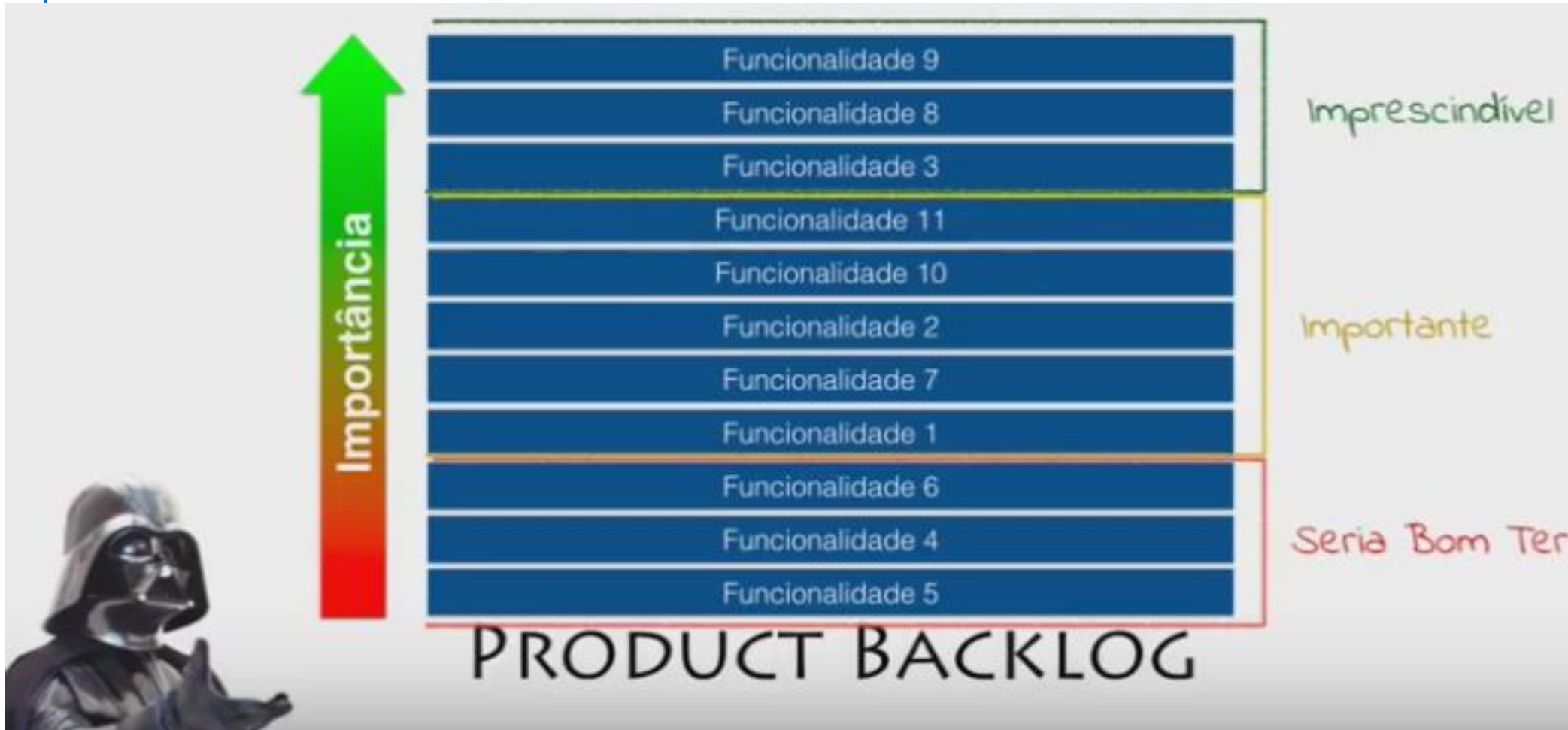
SCRUM - VISÃO GERAL

SCRUM - PRODUCT BACKLOG

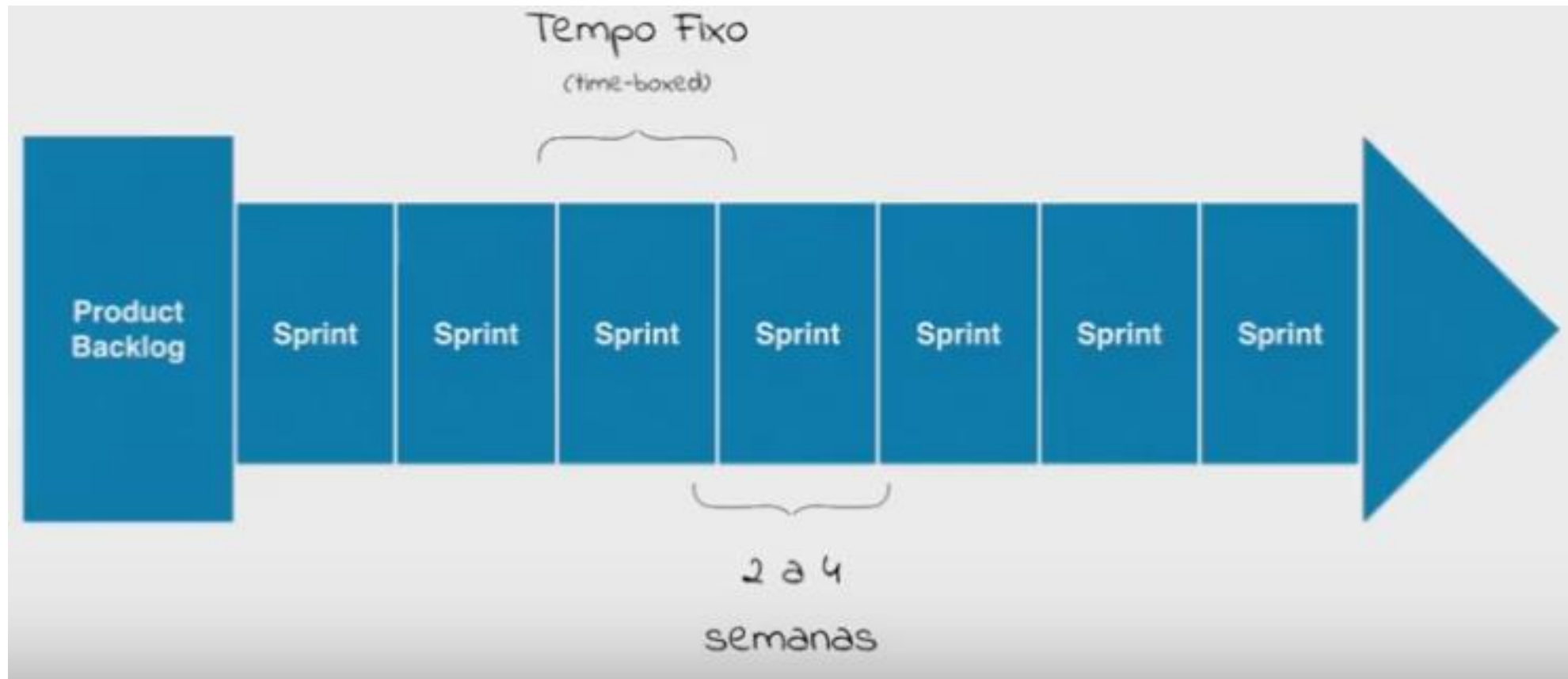
historia de usuário



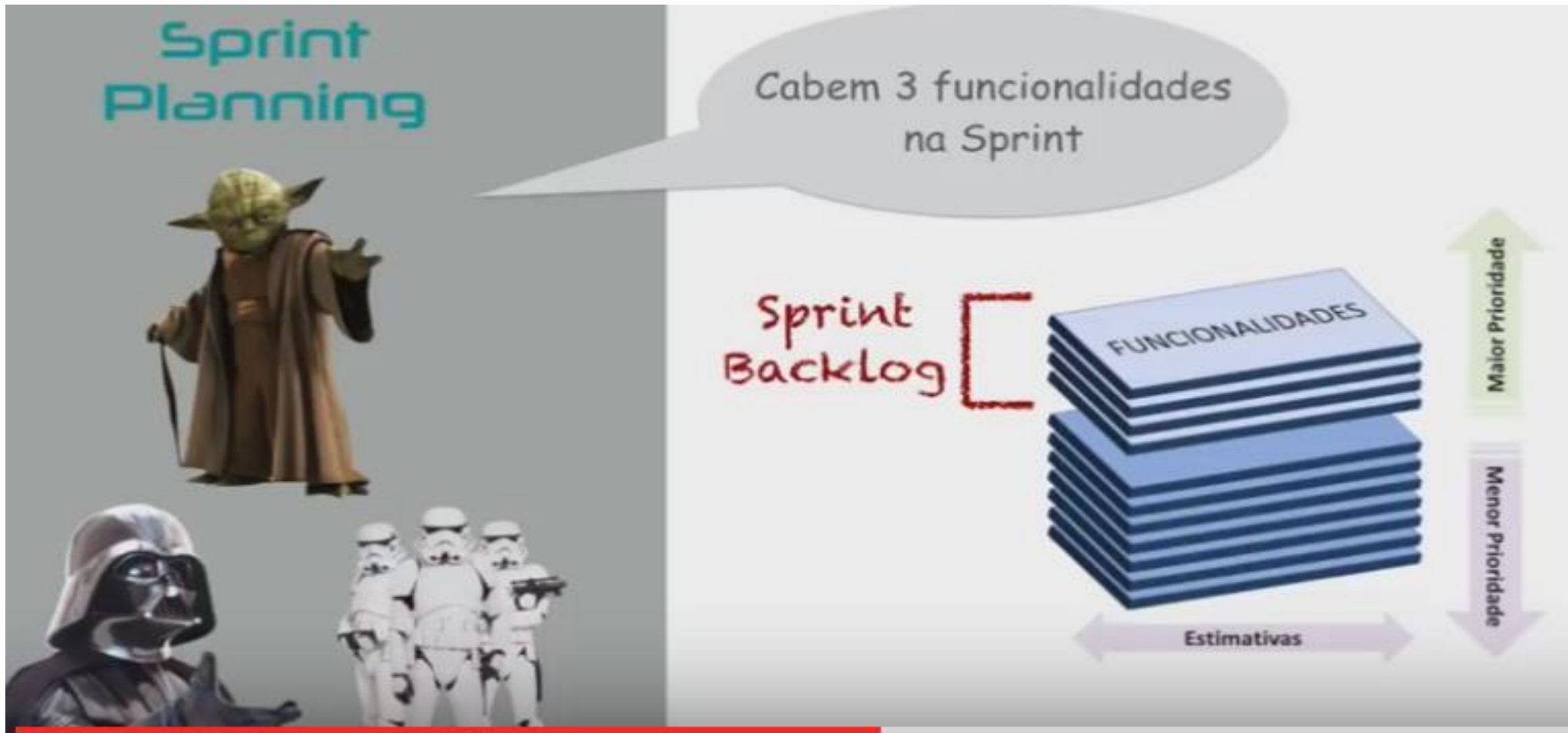
SCRUM - PRODUCT BACKLOG



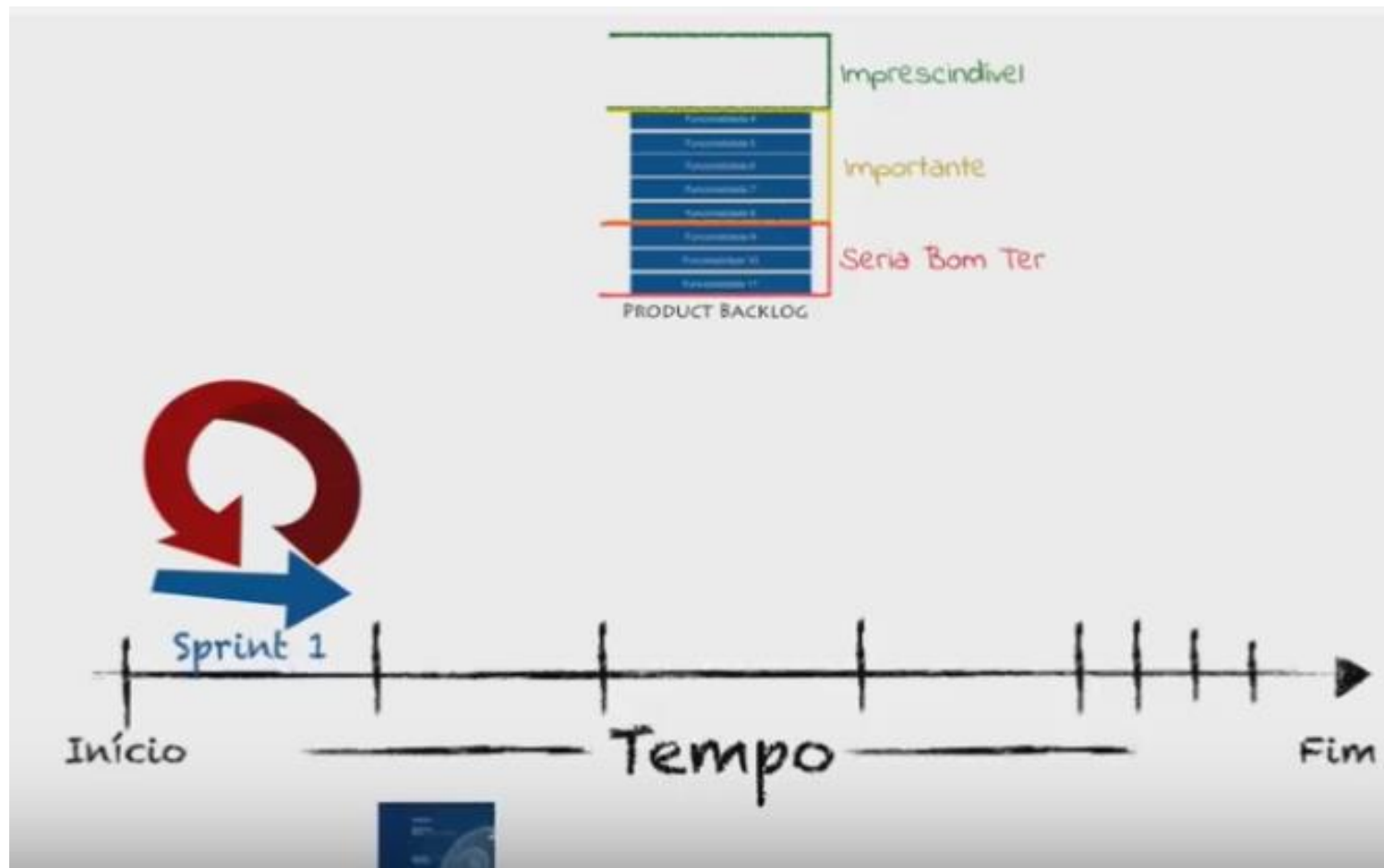
SCRUM - DURAÇÃO DO SPRINT



SCRUM - SPRINT BACKLOG

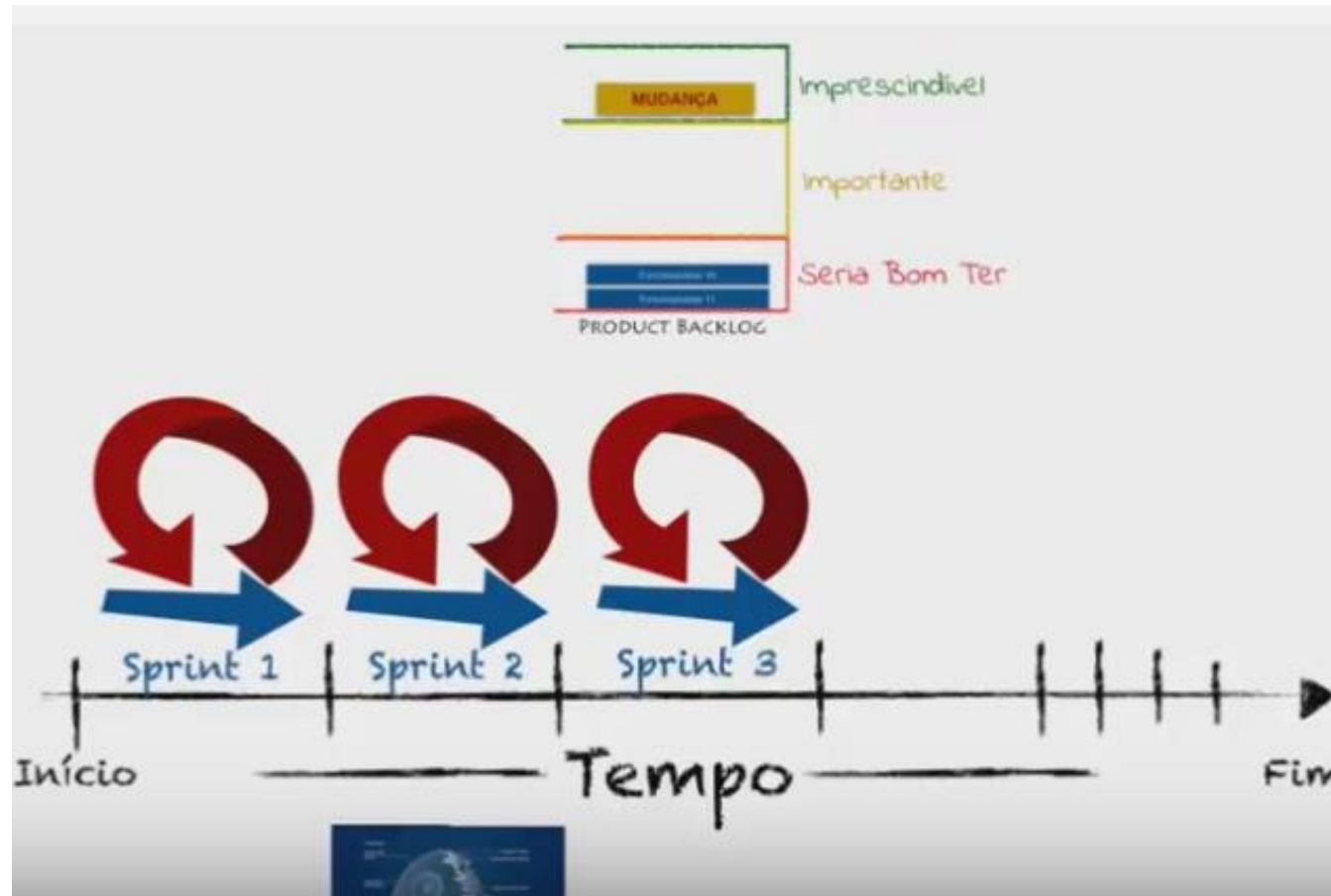


SCRUM - ANDAMENTO DO SPRINT

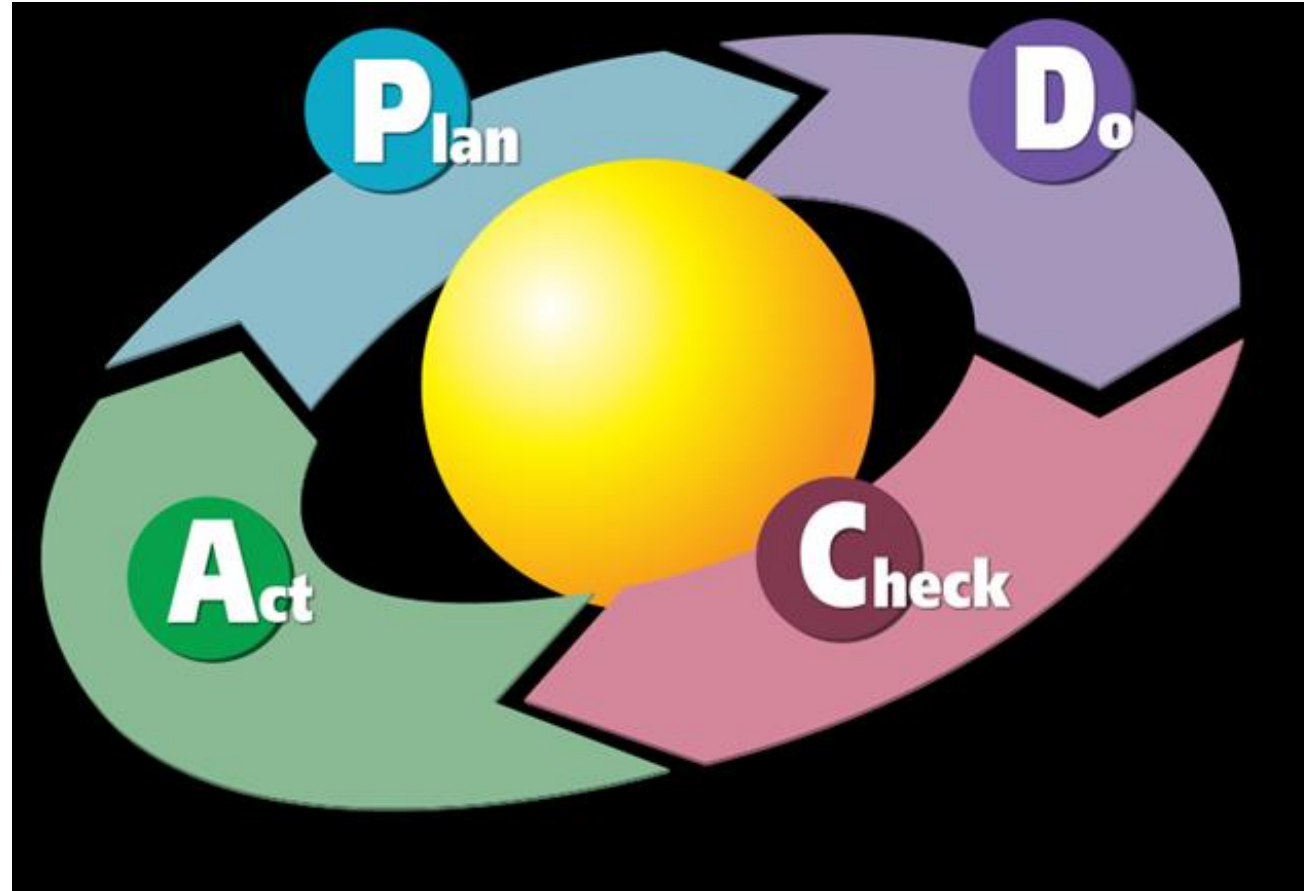


incremento do produto

SCRUM - ANDAMENTO DO SPRINT - MUDANÇAS



SCRUM BASE METODOLÓGICA



O processo não é avaliado **enquanto** está rodando.

SCRUM - COMO FUNCIONA?

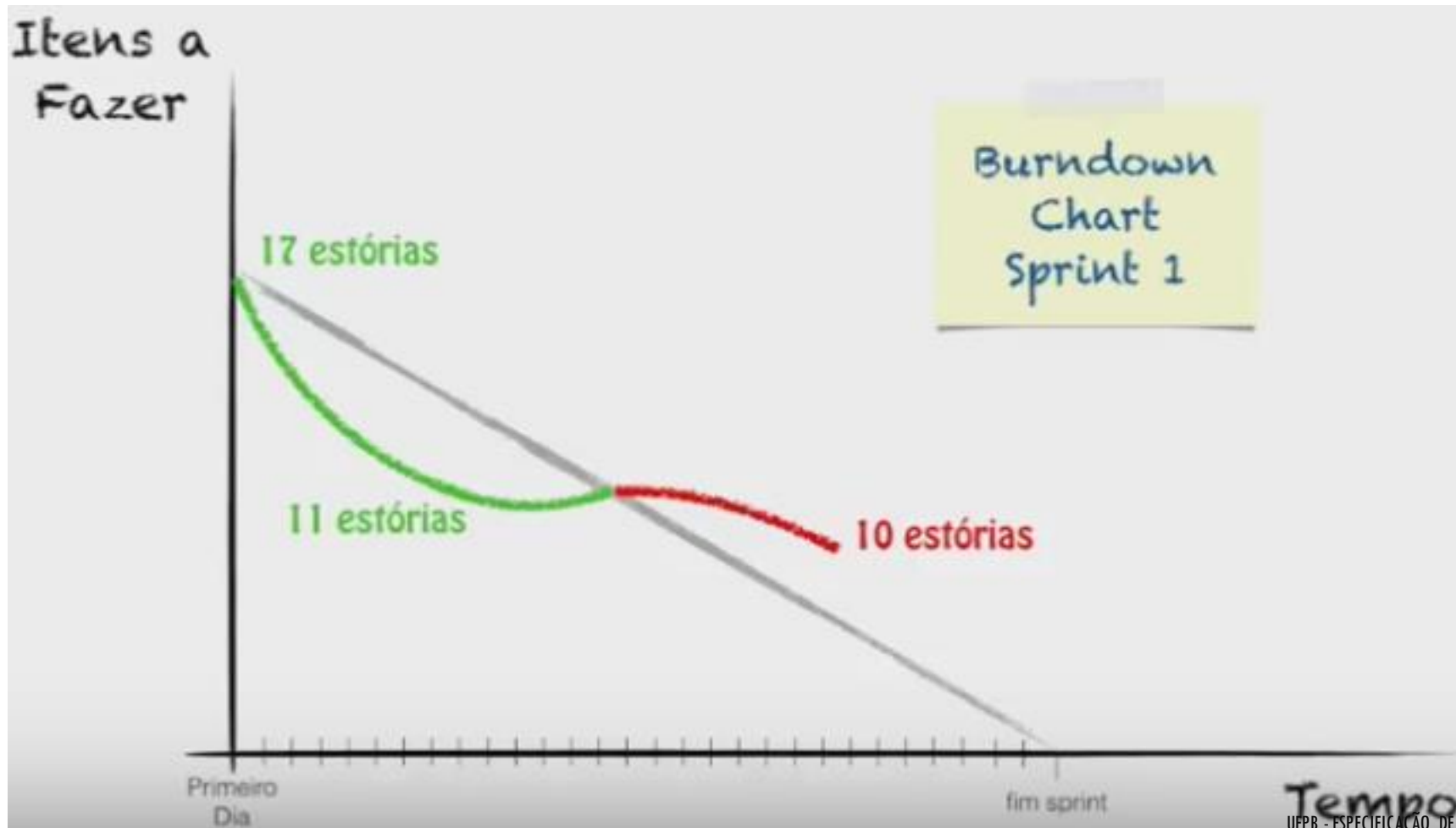
Reuniões Diárias: o *Scrum Master* se reúne diariamente com a *Equipe* num mesmo horário, para que se reporte:

- O que foi feito ontem?

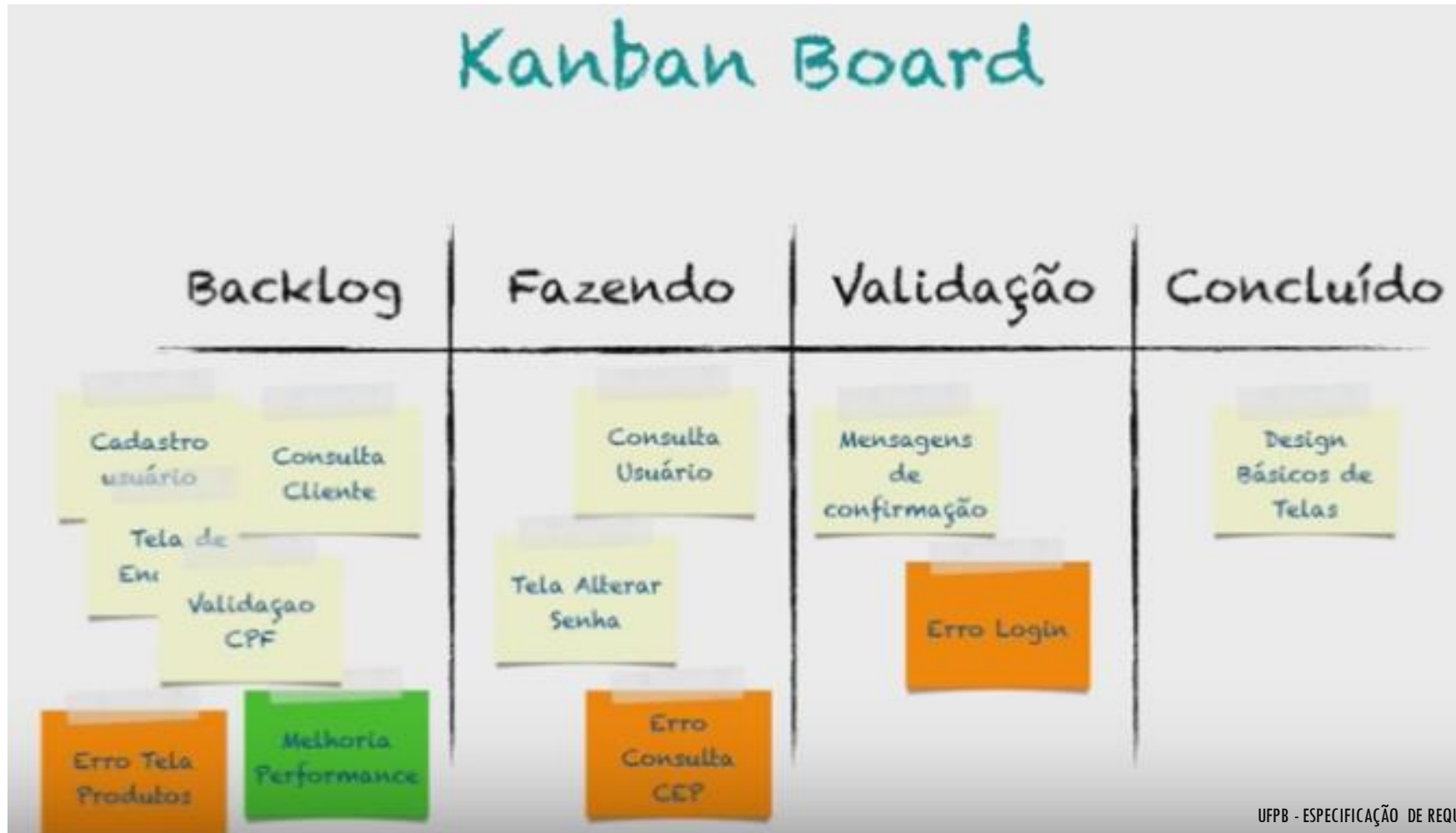
- O que se pretende fazer hoje?

- Quais são os impedimentos que estão atrapalhando a execução das tarefas?

SCRUM - DAILY SPRINT - BURNDOWN CHART



SCRUM - DAILY SPRINT - KANBAN BOARD





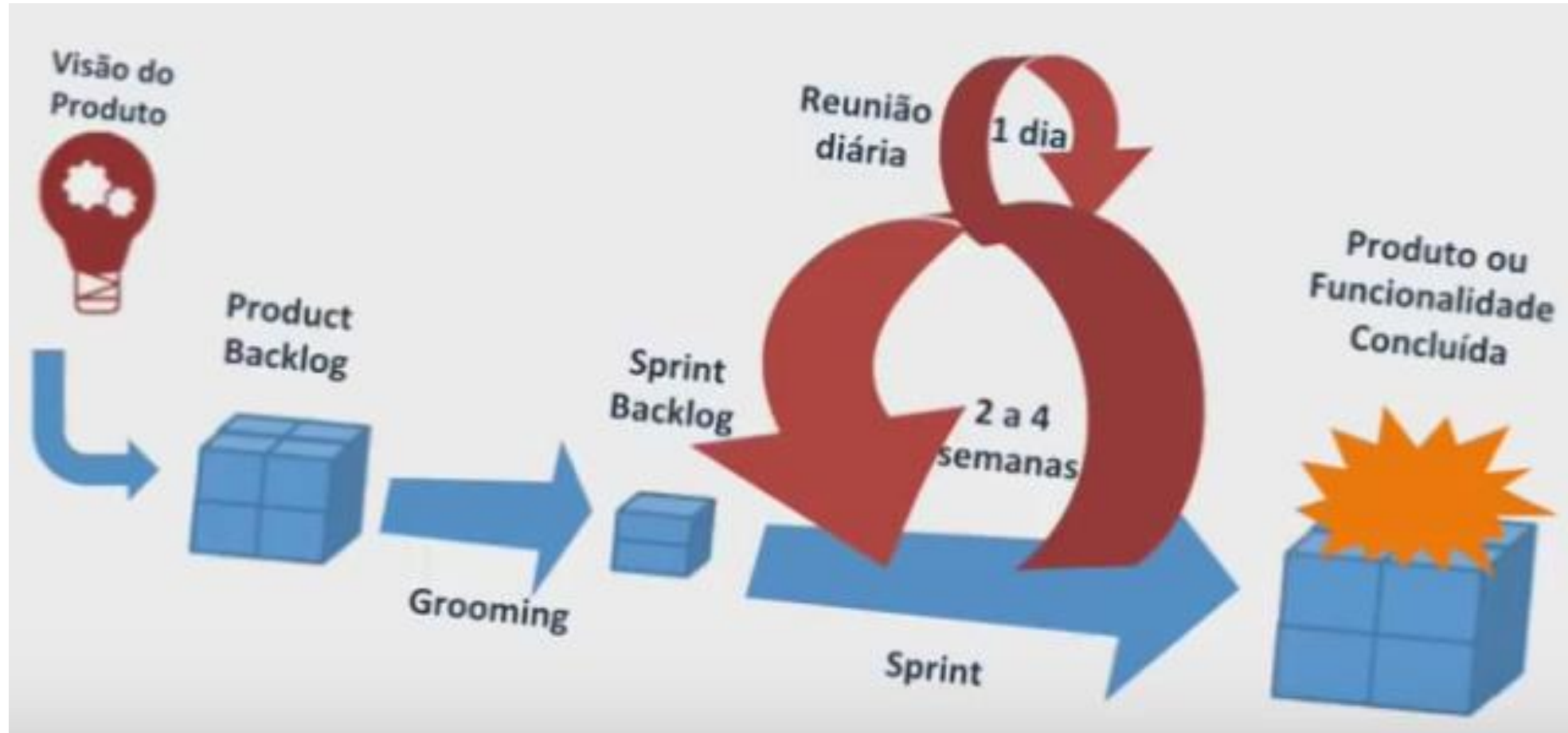
SCRUM - SPRINT REVIEW

Revisões: no final do *Sprint* a *Equipe* demonstra os resultados para o *Product Owner* e demais interessados, de forma que os itens do *Backlog* sejam considerados prontos e então possa se iniciar um novo *Sprint*.

SCRUM - RETROSPECTIVA



SCRUM - CICLO TOTAL



Fase	Processos
Iniciar	1. Criar a Visão do Projeto 2. Identificar o Scrum Master e o(s) Stakeholder(s) 3. Formar o Time Scrum 4. Desenvolver os Épicos 5. Criar o Backlog Priorizado do Produto 6. Conduzir o Planejamento da Release
Planejar e Estimar	7. Criar as Estórias de Usuário 8. Estimar Histórias de Usuário 9. Comprometer Histórias de Usuário 10. Identificar Tarefas 11. Estimar as Tarefas 12. Criar o Backlog do Sprint
Implementar	13. Criar os Entregáveis 14. Conduzir a Reunião Diária 15. Refinamento do Backlog Priorizado do Produto
Revisão e Retrospectiva	16. Demonstrar e Validar o Sprint 17. Retrospectiva do Sprint
Release	18. Enviar os Entregáveis 19. Retrospectiva do Projeto

1..2..3.. TESTANDO



PRÁTICA EM SALA DE AULA

Considerando as etapas do Scrum, vamos aplicá-lo para realizar o seminário da disciplina. Vocês precisarão fazer as seguintes etapas:

1. Criar a visão do produto, no caso de vocês, o produto é um SEMINÁRIO
2. Criar o backlog priorizado para desenvolver o produto (no caso, o produto é seminário desenvolvido, apresentado e avaliado)
3. Conduzir o planejamento dos releases (iterações necessárias para fazer o seminário)
4. Criar um quadro kanban

LEITURA OBRIGATÓRIA

ENGENHARIA DE REQUISITOS EM METODOLOGIAS
ÁGEIS.

Disponível em: <http://www.ailtonsousa.com.br/wp-content/uploads/2014/07/ENGENHARIA-DE-REQUISITOS-EM-METODOLOGIAS-%C3%81GEIS.pdf>

MAIS UMA ATIVIDADE

Tendo como base o artigo recomendado, construir um mapa mental que sintetize de forma clara as ideias chaves discutidas nestes trabalhos como conceitos chaves, temas e seus respectivos relacionamentos.

PRÓXIMA AULA...

Introdução aos requisitos de software





ALGUMA DÚVIDA OU QUESTIONAMENTO?

Danielle Rousy
danielle@ci.ufpb.br

BIBLIOGRAFIA E REFERÊNCIAS

Pressman, Roger S. Engenharia de Software. 6ª. Ed. McGrawHill – Porto Alegre, 2010.

Pleeger, Shari Lawrence. Engenharia de Software: Teoria e Prática. 2ª. Ed. Prentice Hall – São Paulo, 2004. Sommerville, Ian. Engenharia de Software. 8.ª Ed. Addison-Wesley – São Paulo, 2007.

Graham, B. Henderson-Sellers & H. Younessi. The OPEN Process Specification. I.

Catalysis. Disponível em: <http://www.catalysis.org/>

Extreme Programming. Disponível em: <http://www.extremeprogramming.org/>

Agile modeling. Disponível em: <http://www.agilemodeling.com/>

Crystal methodologies. Disponível em: <http://www.crystalmethodologies.org/>

AmbySoft: www.ambysoft.com

Agile Data: www.agiledata.org

Larman, Craig. Agile and Iterative Development. Addison-Wesley, 2003

Jacobson, I. and Booch, G. and Rumbaugh, J. *The Unified Software Development Process*. Capítulos 1 a 5.

Kruchten, P. *The Rational Unified Process – an Introduction*.

MAIS INFORMAÇÕES

Agile Alliance - www.agilealliance.org

- Ótima fonte sobre métodos ágeis

Scrum Alliance - www.scrumalliance.org/

Mountain Goat Software

- www.mountaingoatsoftware.com
- Site de um treinador de *Scrum Masters*

Site do Ken Schwaber - www.controlchaos.com